

第7章 数据库设计

7.4 逻辑结构设计



□ 基本任务

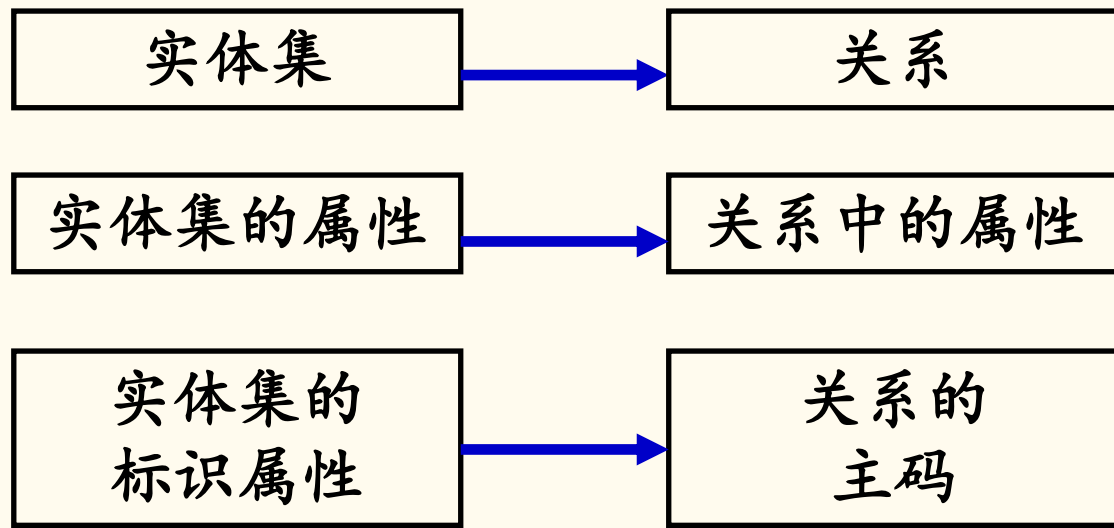
- 将前一阶段设计得到的概念数据模型(**EE-R模型**)转换成用户所选择的数据库管理系统(**DBMS**)支持的逻辑数据模型。
- 由于在概念设计阶段选择的是**EE-R模型**，而目前最流行的是关系数据库系统，因此，数据库的逻辑设计的基本任务就是：
 - 将**EE-R模型**转换成等价的关系数据库模式

数据库的逻辑结构设计

□ **基本方法：**每个‘实体集’被转换成一个关系，每个‘联系’通常也被转换成一个关系

□ 实体集的转换方法

- 每个实体集被转换成一个关系(模式)
- 关系及其属性的命名采用原实体集及其属性的名称



□ 联系的转换方法

- 在一般情况下，每个联系也被转换成一个关系模式，联系名被用作转换得到的关系模式的关系名，该关系模式中的属性由两部分组成：
 - 联系自身所具有的属性
 - 与该联系相关的实体集的标识属性
- 来自于相关实体集的标识属性也是相关实体集转换得到的关系模式的主码，因此它们也是联系转换得到的关系模式中的外码

❑ 在从概念数据模型（EE-R）到逻辑数据模型（关系模型）的转换过程中，还可能存在一些需要处理的问题。

1. 命名与属性域的处理
2. 非原子属性处理
3. 联系的转换
4. ISA联系的转换
5. 规范化
6. RDBMS性能调整
7. 约束条件设置
8. 关系视图的设计

1. 命名与属性域的处理

□ 关系及属性的命名

- 尽量采用在**EE-R**模型中原有的名称
- 可以重新命名，但要避免命名的冲突现象
 - 在同一个数据库模式中，关系名具有唯一性
 - 在同一个关系模式中，属性名具有唯一性

□ 属性域的定义

- 根据**DBMS**的选型进行必要的数据类型转换

规则1：实体集到关系的转换

- ❑ 每个实体集都被单独转换成一个独立的关系。

E-R concepts	Relational Database
entity name	table name
attribute name	column name
the single-valued attributes	columns of the table
the composite attributes	multiple simple columns
an identifier for the entity	a candidate key for the table
primary identifier	primary key
entity instance	row

- 转换后得到的table和column的命名，可以直接使用对应实体的实体名和属性名；
- composite attribute将被替换为它的所有成员属性；
- 在转换后的table中，如果存在column的‘重名’，可以对column进行重命名。

2. 非原子属性的处理

❑ 组合属性 (composite attribute)

- 将组合属性横向展开成多个属性（参见前面的转换规则1）

❑ 多值属性 (multi-valued attribute)

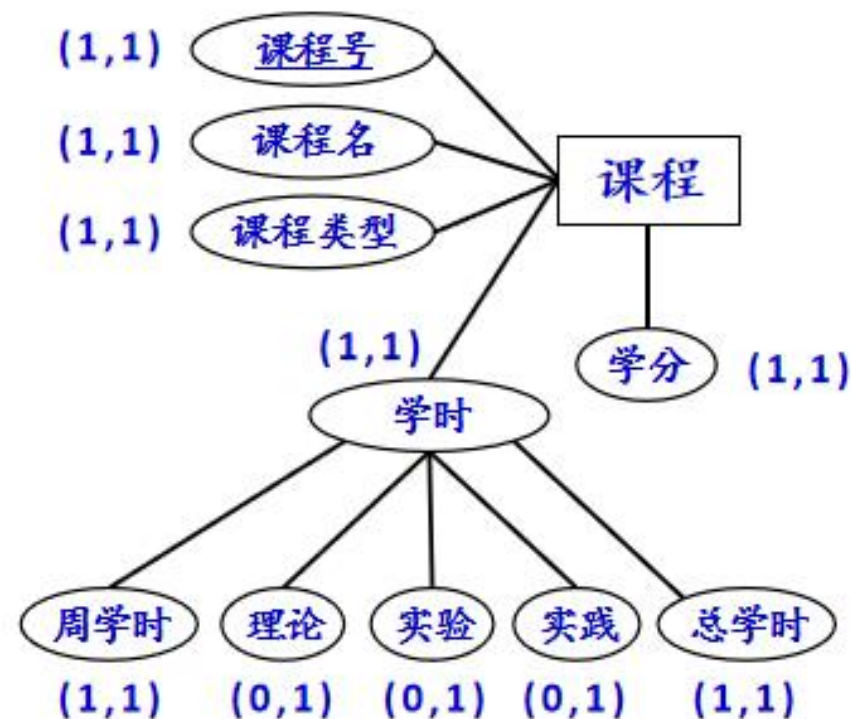
非原子属性（组合属性）的转换方法 1

❑ 例：如右图所示的‘课程’实体集

❑ 按照规则1，将该实体集转换为如下的关系

‘课程’关系

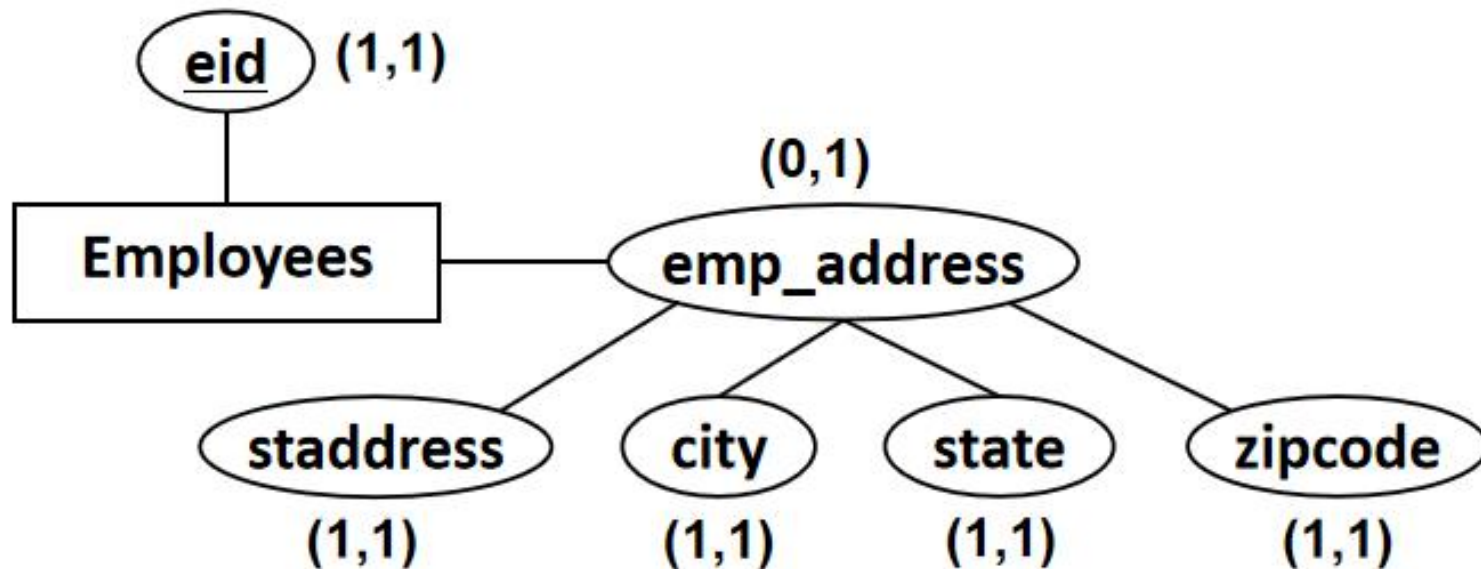
属性	属性级约束
课程号	primary key
课程名	NOT NULL
课程类型	NOT NULL
学分	NOT NULL
周学时	NOT NULL
理论学时	
实验学时	
实践学时	
总学时	NOT NULL



- 根据规则，将用周学时、理论学时...等成员属性，代替组合属性‘学时’出现在关系中；
- 这些成员属性上约束，需根据原组合属性及各个成员属性上的属性基数约束来决定。

非原子属性（组合属性）的转换方法 2

❑ 例：设有如下所示职工实体集Employees，转换成关系模型后的属性约束如下



关系 Employees

属性	属性级约束
eid	primary key
staddress	
city	
state	
zipcode	

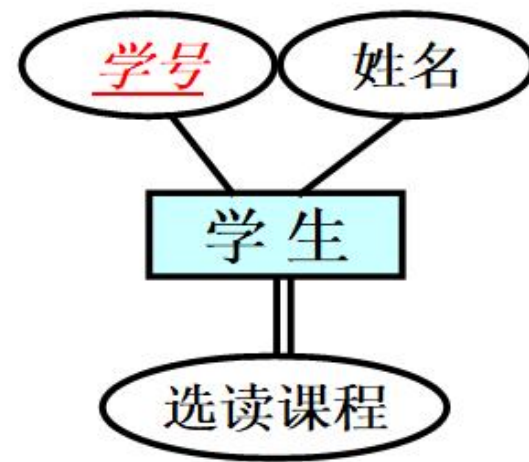
- ❑ 根据在组合属性emp_address上定义的基数约束，在转换后的关系中，在staddress、city、state、zipcode这四个属性上，要么全为空值、要么都不为空值；
- ❑ 这样的约束条件，无法在属性级去定义，但可以通过表级CHECK约束来定义。

非原子属性（多值属性）的转换方法 1

□ 例：一个简单的学生实体集，‘选读课程’是一个多值属性

学号	姓名	选读课程
S1307	王承志	Database Operating System Computer Network

（学生实体）



非原子属性（多值属性）的转换方法 2

□多值属性的处理方法一

- 关系模式不变，但原有关系的一个元组将被纵向展开成多个元组（如下表所示学生关系一，满足1NF的要求）
- 在上述转换过程中，虽然实体集(或关系模式)中的属性没有增加，但转换得到的关系模式的码由**原实体集的主标识符属性**和**该多值属性**联合构成（红色的属性名集合）。

学号	姓名	选读课程
S1307	王承志	Database
S1307	王承志	Operating System
S1307	王承志	Computer Network

‘学生’关系一

- ‘学生’关系一最高只能满足1NF

- 经过关系规范化设计，可以将左边的‘学生’关系一模式分解为右边的‘学生’关系二和‘选课’关系。

□多值属性的处理方法二

- 每一个多值属性，都将被分解出去，和原实体集的主标识符属性联合起来单独组成一个新的‘选课’关系。
- ‘选课’关系的码由**原实体集的主标识符属性**和**该多值属性**联合构成。
- 其他属性按照规则1转换成如下所示的‘学生’关系二。

学号	姓名
S1307	王承志

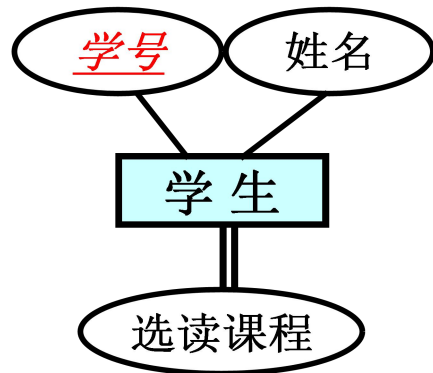
‘学生’关系二

学号	选读课程
S1307	Database
S1307	Operating System
S1307	Computer Network

‘选课’关系

规则2：多值属性向关系的转换

E-R concepts	Relational Database
the plural multi-valued attribute	name for the new table
primary identifier	column for the new table
the multi-valued attribute	column for the new table
primary identifier + the multi-valued attribute	primary key
entity instance + an element of the multi-valued attribute	row



学生之 EE-R 图



学号	姓名
S1307	王承志

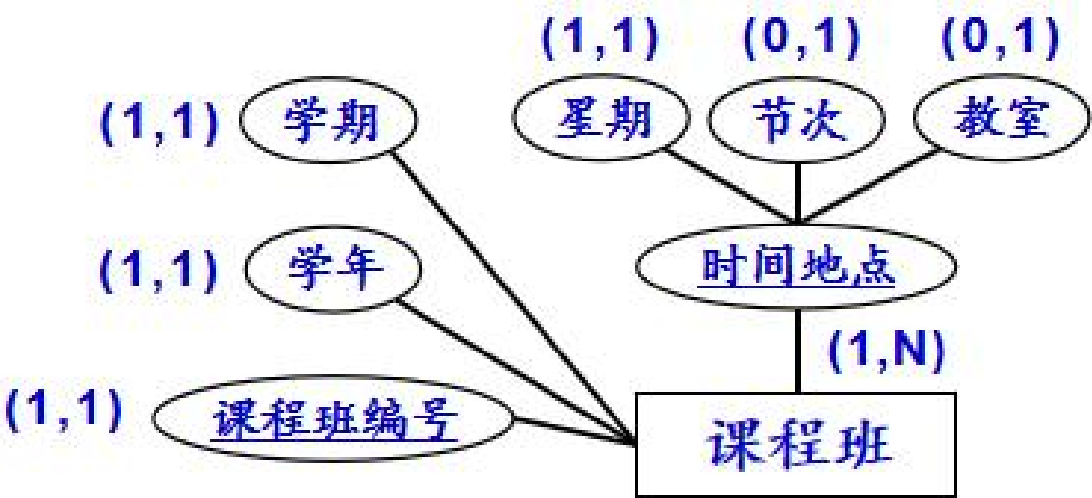
‘学生’关系

学号	选读课程
S1307	Database
S1307	Operating System
S1307	Computer Network

‘选课’关系

非原子属性（多值属性+组合属性）的转换方法 3

例：如右图所示的‘课程班’实体集



按照规则1&规则2转换为如下的两个关系

‘课程班’ 关系

属性	属性级约束
课程班编号	primary key
学年	NOT NULL
学期	NOT NULL

‘课程班时间地点’ 关系

属性	属性级约束
课程班编号	primary key
星期	primary key
节次	primary key
教室	primary key

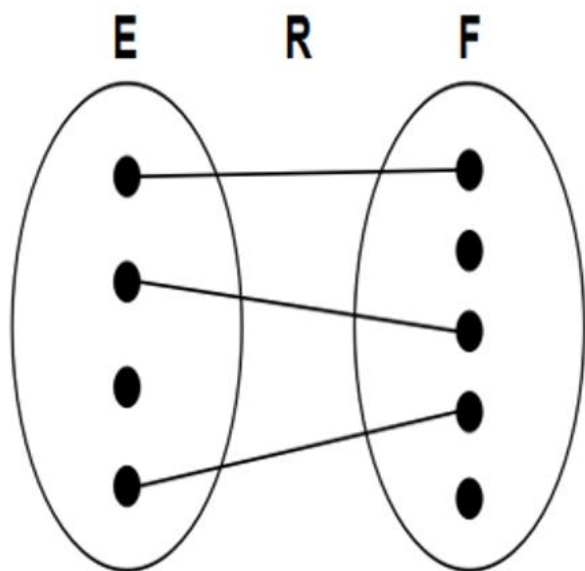
说明：根据转换规则确定的各个关系上的primary key的定义，不一定正确，需要再经过关系规范化设计来进行验证、纠错、补充。

3. 联系的转换

- 一般情况下，一个联系可以被转换成一个关系。
- 但是在有些情况下，联系也可被归并到相关联的实体集所对应的关系模式中去，即将联系与某个(或几个)相关联的实体集共同转换成一个关系模式。
- 下面以两个实体集之间的二元联系为例，介绍在不同情况下其逻辑设计的方法。
 - 两个实体集之间的二元联系，可以根据该联系的函数关系，以及每个实体参与该联系的参与方式将其转换成1、2或3个关系模式
 - 根据实体在联系中的最小参与基数，可以将参与方式分为
 - $\min_card(E, R) = 1$
 - 称实体E在联系R中是‘强制参与’(mandatory participation)，又被称为是‘全参与’
 - $\min_card(E, R) = 0$
 - 称实体E在联系R中是‘可选参与’(optional participation)，又被称为是‘非全参与’

实体集与联系的 ‘全参与’ 与 ‘非全参与’

- 在实体与联系之间的‘参与基数’中，包含着一对一、一对多、多对多等‘函数关系’，后面将综合考虑‘函数关系’和‘参与基数’来定义联系到关系的转换规则。

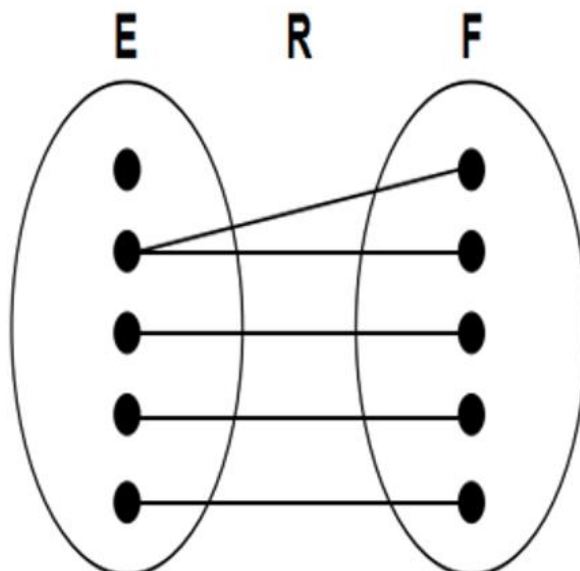


(a) 一对一

$$\text{card}(E,R) = (0,1)$$

$$\text{card}(F,R) = (0,1)$$

都是‘非全参与’

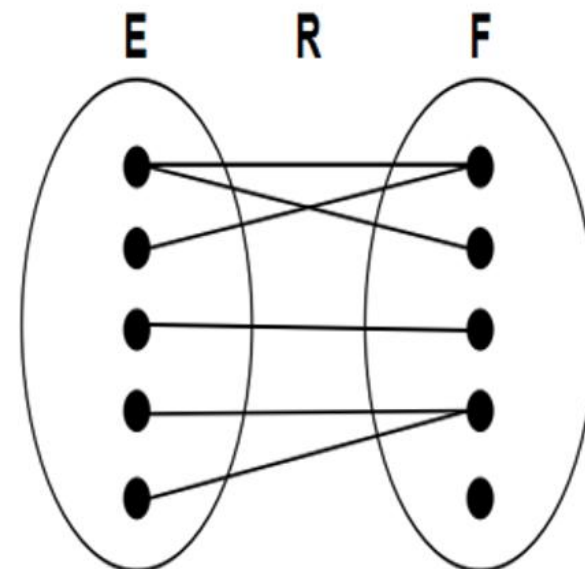


(b) 一对多

$$\text{card}(E,R) = (0,N)$$

$$\text{card}(F,R) = (1,1)$$

E 是‘非全参与’
F 是‘全参与’



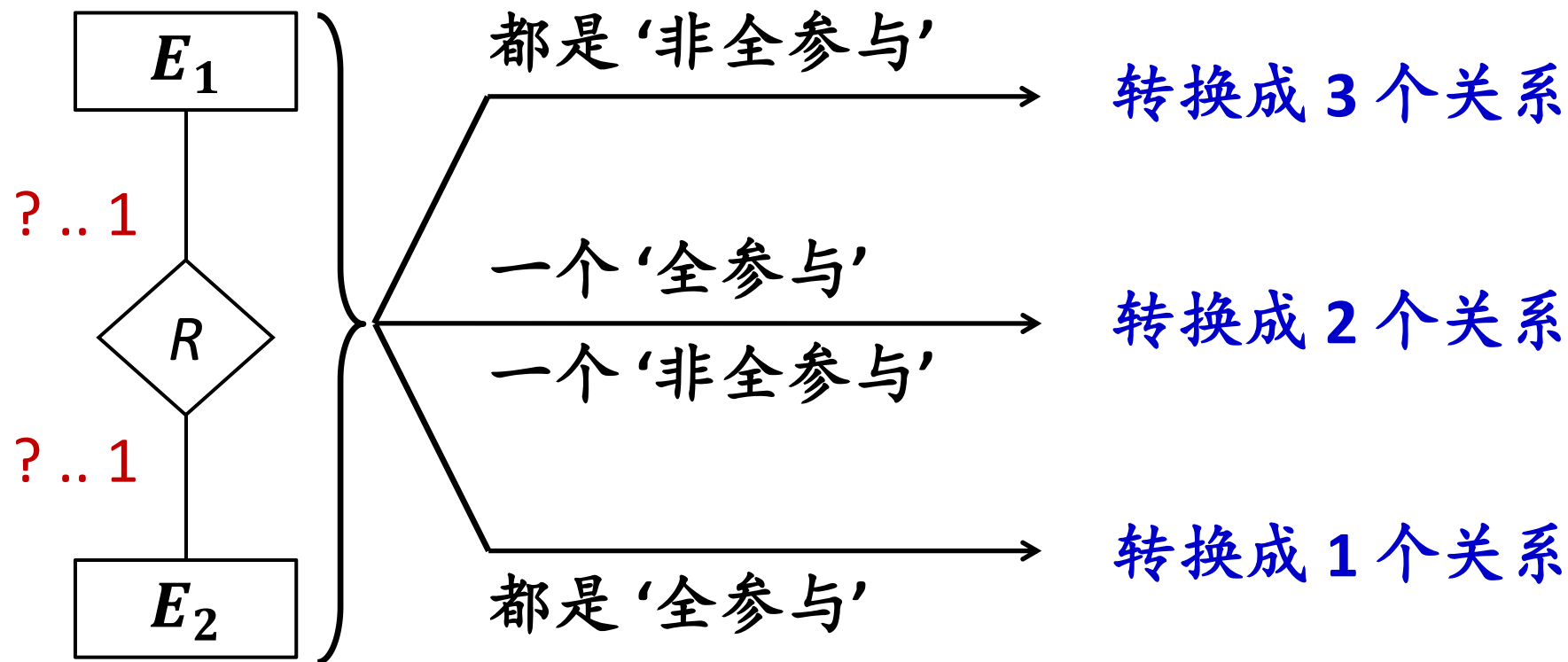
(c) 多对多

$$\text{card}(E,R) = (1,N)$$

$$\text{card}(F,R) = (1,N)$$

都是‘全参与’

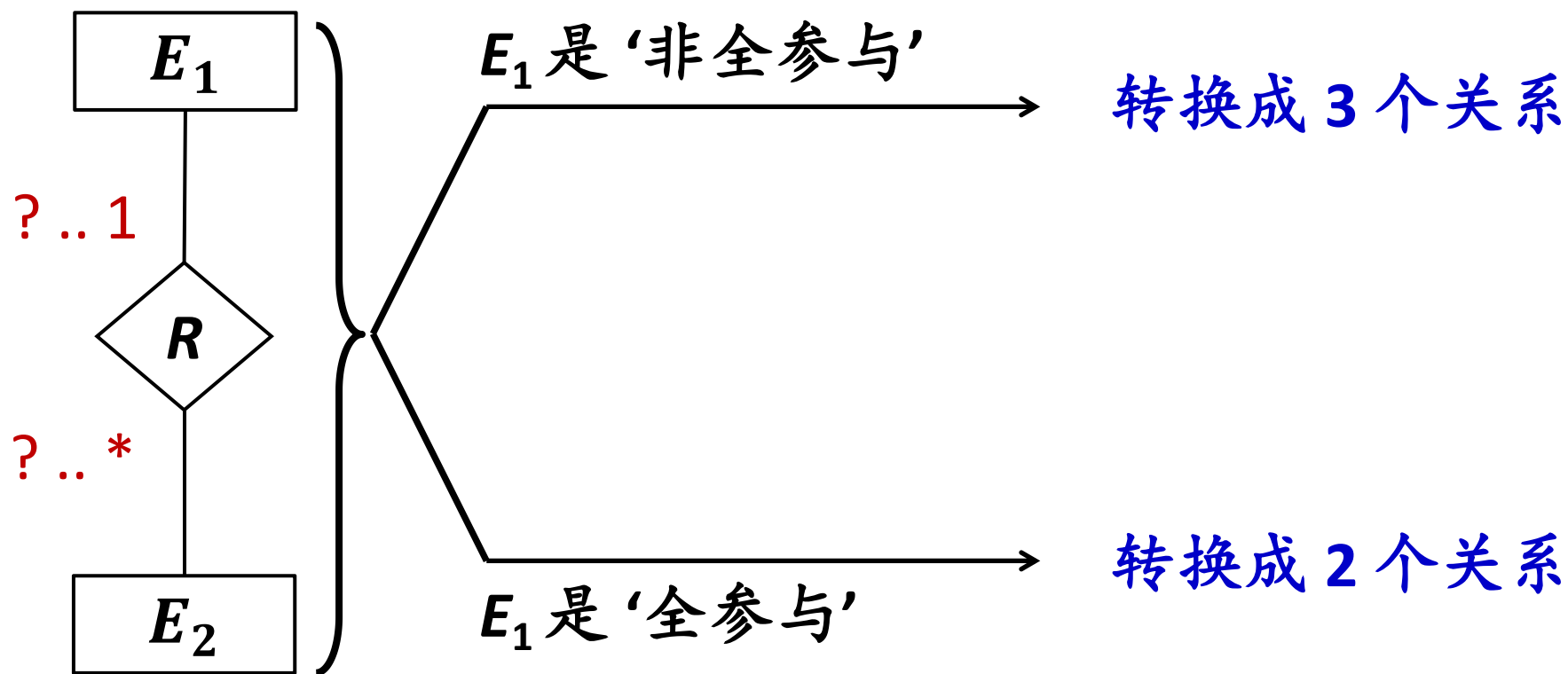
联系的转换 1



‘一对一’联系的转换方式

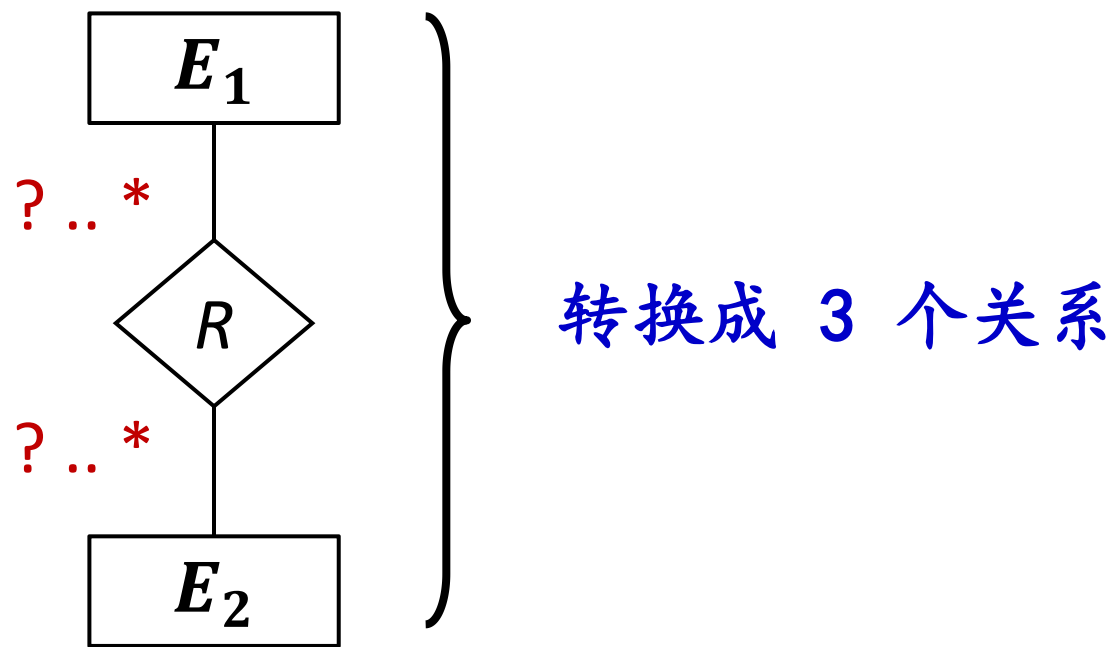
(? 代表 0 或 1)

联系的转换 2



‘多对一’联系的转换方式

联系的转换 3



‘多对多’联系的转换方式

规则3：“一对一”二元联系的转换 1

① 一对一：都是‘非全参与’（图1）

- 将被转换成三个关系模式。其中， $k1$ 、 $k2$ 分别是实体集 E_1 和 E_2 的标识符(identifier)

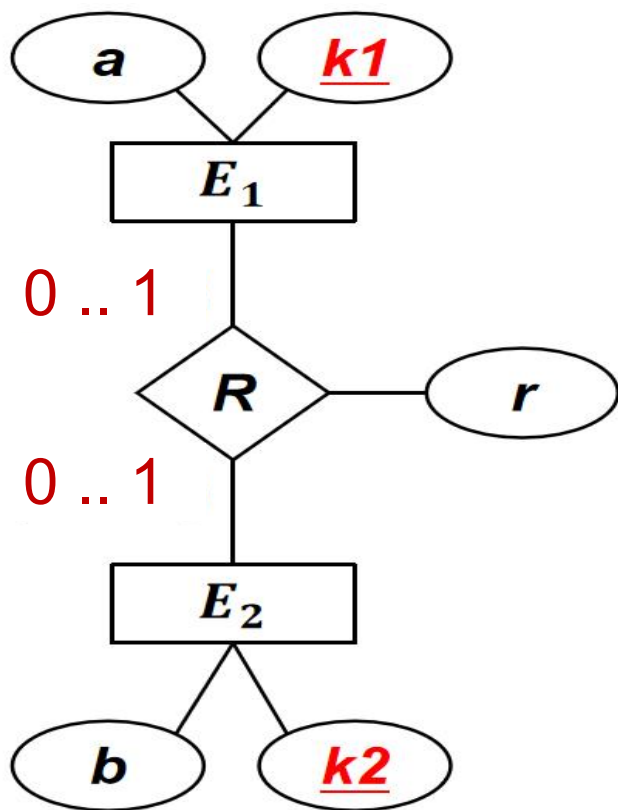


图1 一对一联系

$E_1(k1, a)$

- 码是 $k1$

$E_2(k2, b)$

- 码是 $k2$

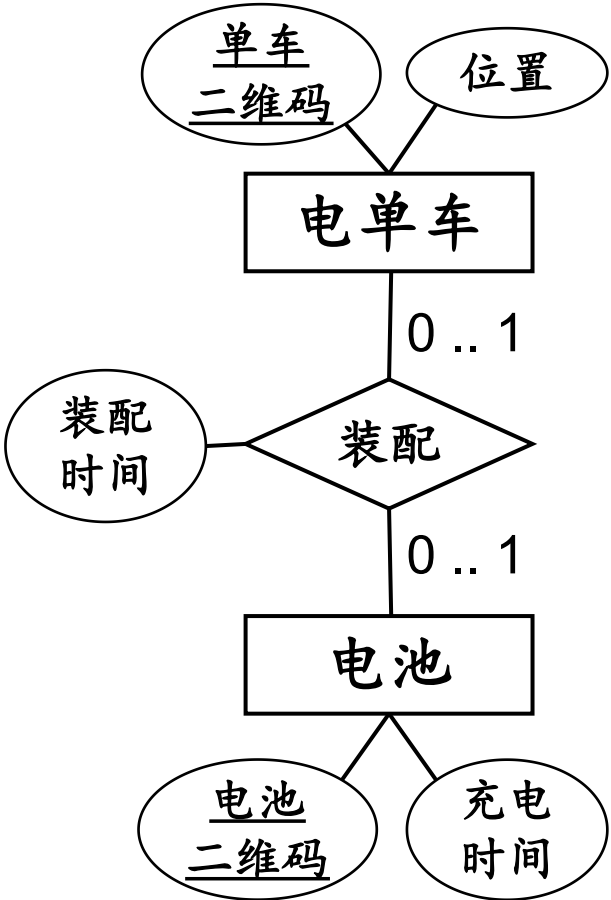
$R(k1, k2, r)$

- 两个码： $k1$, $k2$
- $k1$ 和 $k2$ 也是关系 R 的两个外码

规则3：“一对一”二元联系的转换 1（示例）

① 一对一（共享电单车与电池）：都是‘非全参与’

➤ 一辆电单车最多安装一块电池，一块电池最多只能被安装在一部电单车上



转换结果如下：

关系名	属性集	候选码	外码
电单车	单车二维码，位置	单车二维码	
电池	电池二维码，充电时间	电池二维码	
装配	单车二维码 电池二维码 装配时间	单车二维码	单车二维码
		电池二维码	电池二维码

【思考】如果将其转换为如下的单个关系，会存在哪些问题？
电单车(单车二维码, 位置, 电池二维码, 充电时间, 装配时间)

规则3：“一对一”二元联系的转换 2

② 一对一： E_1 是‘全参与’， E_2 是‘非全参与’

➤ 可以将联系 R 与实体集 E_1 合并，将图2转换成二个关系模式。

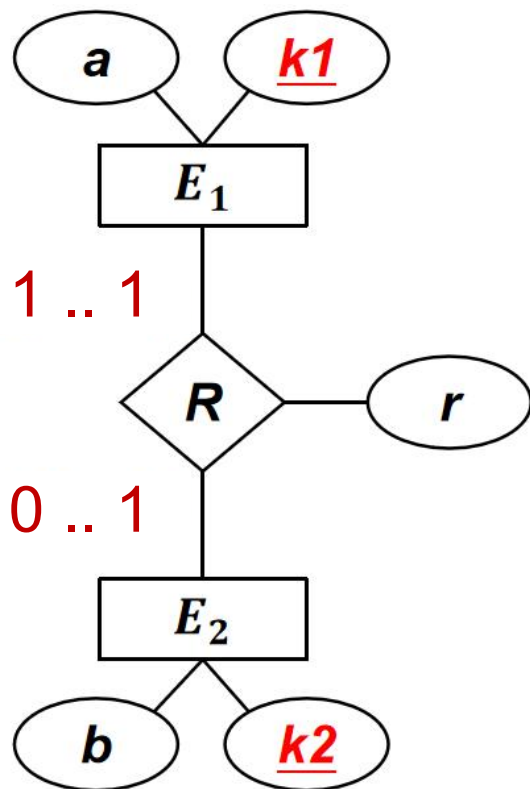


图2 一对一联系

$E_1 (k1, a, k2, r)$

- 码是 $k1$
- $k2$ 是关系 E_1 的外码

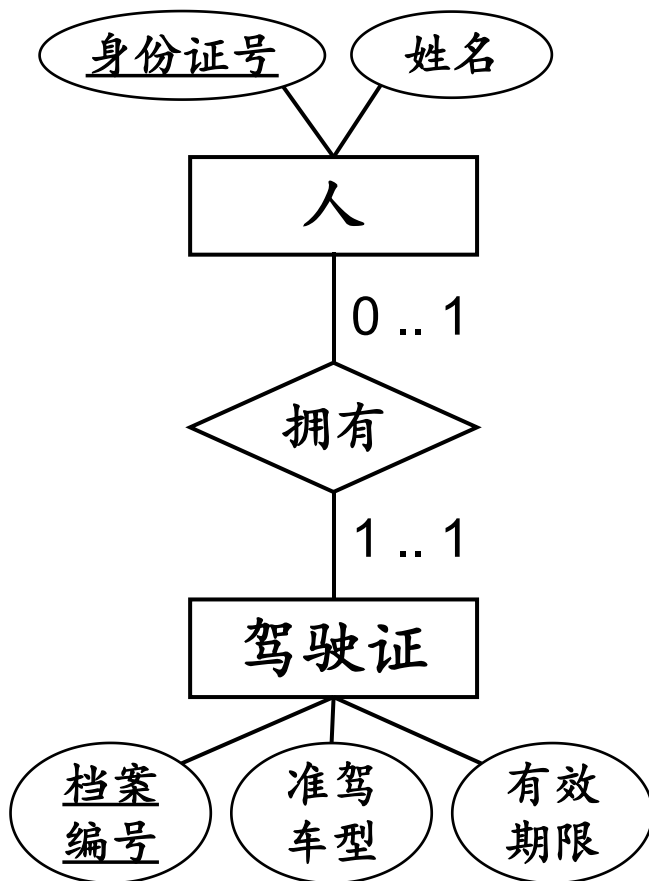
$E_2 (k2, b)$

- 码是 $k2$

规则3：“一对一”二元联系的转换 2（示例）

② 一对一（人与驾驶证）：驾驶证是‘全参与’，人是‘非全参与’

□ 转换结果如下：



关系名	属性集	候选码	外码
人	身份证号, 姓名	身份证号	
驾驶证	档案编号, 准驾车型 有效期限, 身份证号	档案编号	身份证号

【思考】

- 如果将其转换为如下的单个关系，会存在哪些问题？
人（身份证号, 姓名, 档案编号, 准驾车型, 有效期限）
- 如果将其转换为如下的三个关系，会存在哪些问题？
人（身份证号, 姓名）
驾驶证（档案编号, 准驾车型, 有效期限）
拥有（身份证号, 档案编号）

规则3：“一对一”二元联系的转换 3

③ 一对一：都是‘全参与’

➤ 将三者全部合并，将图3转换成一个关系模式。

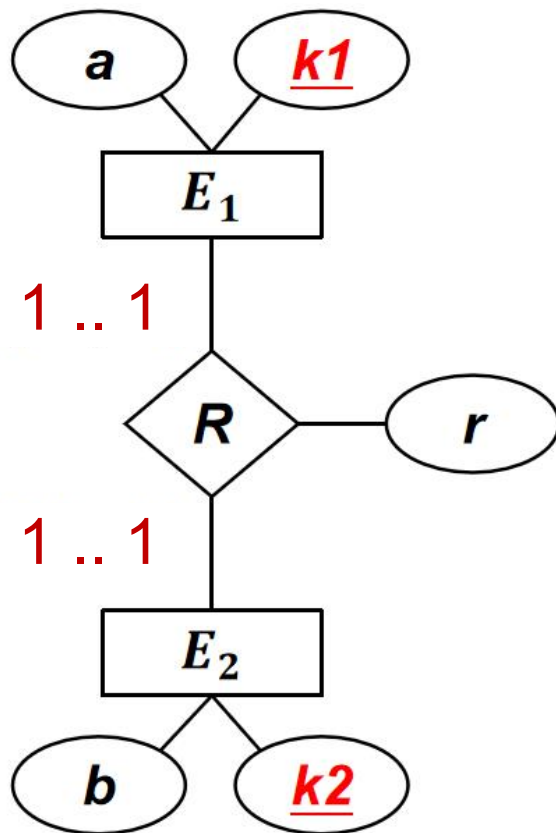


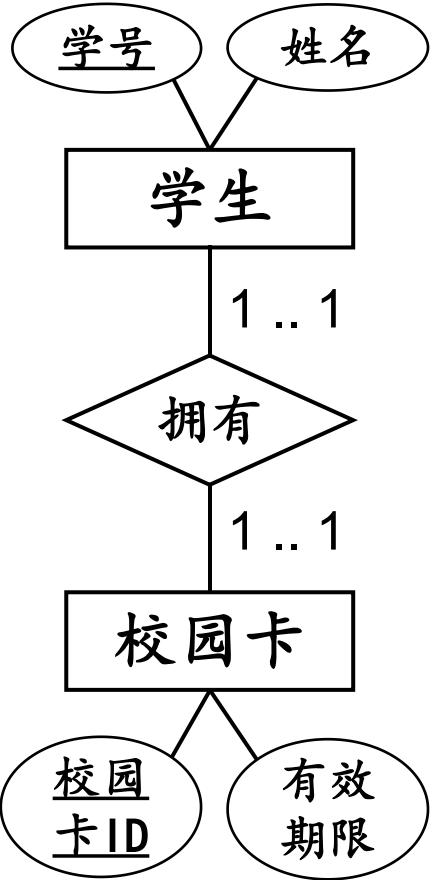
图3 一对一联系

$E(k1, a, k2, b, r)$

- $k1, k2$ 是该关系的两个候选码

规则3：“一对一”二元联系的转换 3（示例）

③ 一对一（学生与校园卡）：都是‘全参与’



转换结果如下：

关系名	属性集	候选码	外码
学生	学号, 姓名, 校园卡ID, 有效期限	学号	
		校园卡ID	

【思考】如果转换为如下的两个或三个关系，会存在哪些问题？

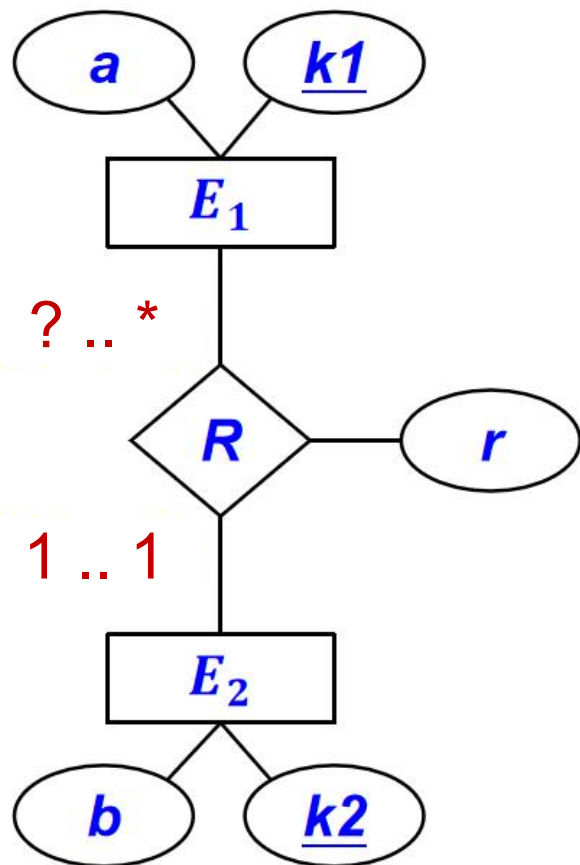
学生（学号，姓名，校园卡ID）

校园卡（校园卡ID，有效期限）

规则4：“一对多” 二元联系的转换 1

④ 一对多：(单值参与) 多端 E_2 是‘全参与’

➤ 可以将联系 R 与实体集 E_2 合并，将图4转换成 2 个关系模式



$E_1(k1, a)$

- 码是 $k1$

$E_2(k2, b, k1, r)$

- 码是 $k2$
- $k1$ 是关系 E_2 的外码

图4 一对多联系

规则4：“一对多” 二元联系的转换 2

⑤ 一对多：(单值参与) 多端 E_2 是‘非全参与’

➤ 必须将图5转换成3个关系模式

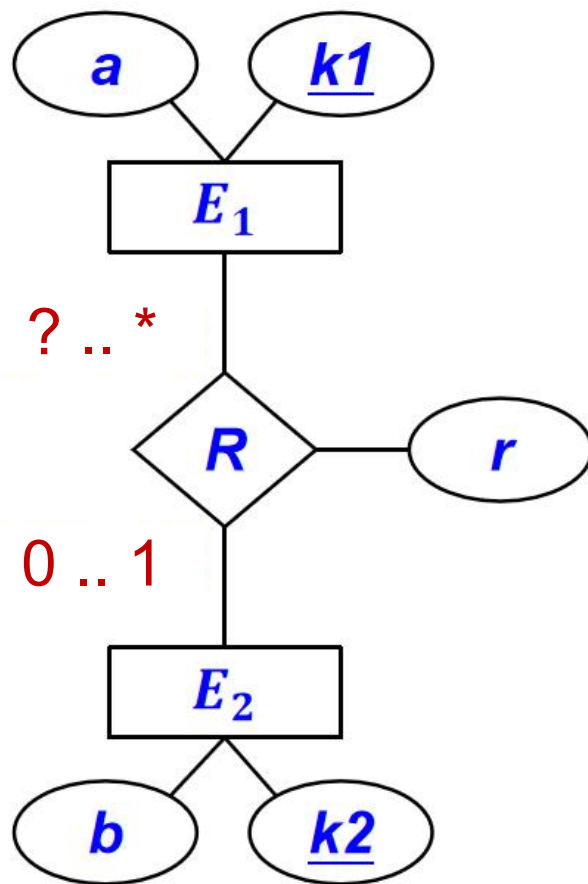


图5 一对多联系

$E_1(k1, a)$

- 码是 $k1$

$E_2(k2, b)$

- 码是 $k2$

$R(k2, k1, r)$

- 码是 $k2$
- $k1$ 是关系 R 的外码

□ 注意：

- 在这里不能合并关系 E_1 与关系 R
- 也可将关系 E_2 与关系 R 关系合并（合并具有相同码的关系）

规则5：“一对多” 二元联系的转换

⑥ 多对多：将转换成3个关系模式

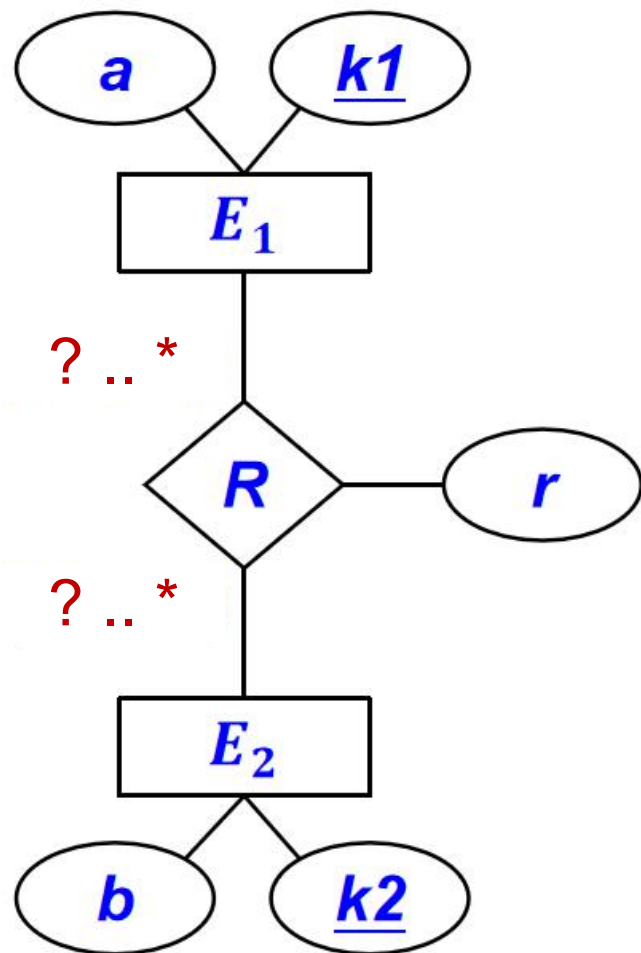


图6 多对多联系

$E_1 (k1, a)$

- 码是 $k1$

$E_2 (k2, b)$

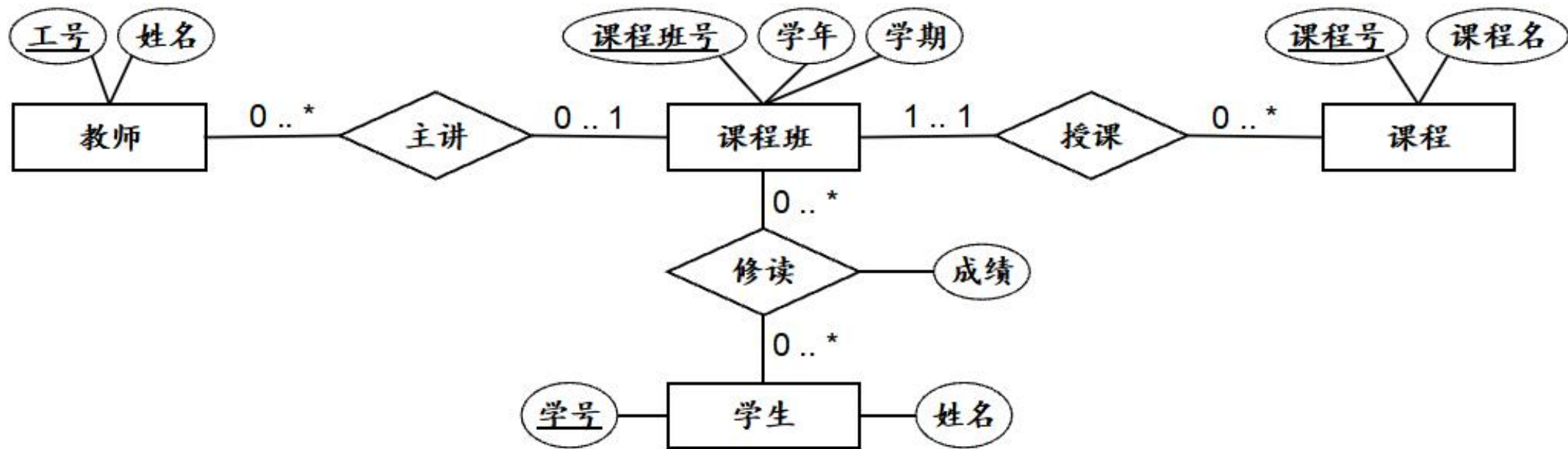
- 码是 $k2$

$R (k1, k2, r)$

- $(k1, k2)$ 联合构成关系 R 的码
- $k1$ 和 $k2$ 是关系 R 的两个外码

“一对多” & “多对多” 二元联系的转换（示例）

□ 设有如下图所示的ER模型



□ 根据规则4，可将‘授课’联系合并到‘课程班’关系中去，‘主讲’联系可单独转换，也可被合并到‘课程班’关系中；根据规则5，‘修读’联系必须单独转换成一个关系。

□ 转换后的关系模式如下（带下划线的是各个关系的码）

教师(工号, 姓名)

课程(课程号, 课程名)

学生(学号, 姓名)

课程班(课程班号, 学年, 学期, 课程号)

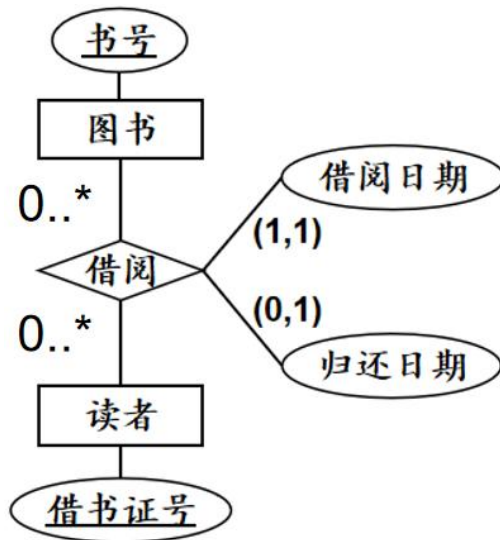
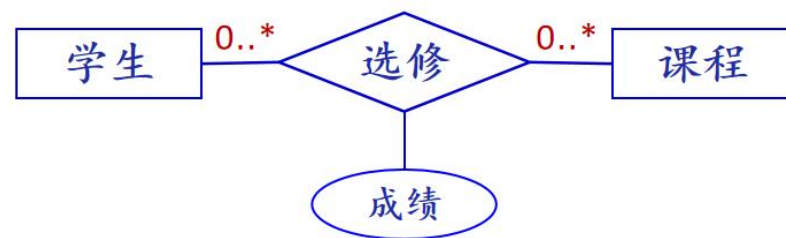
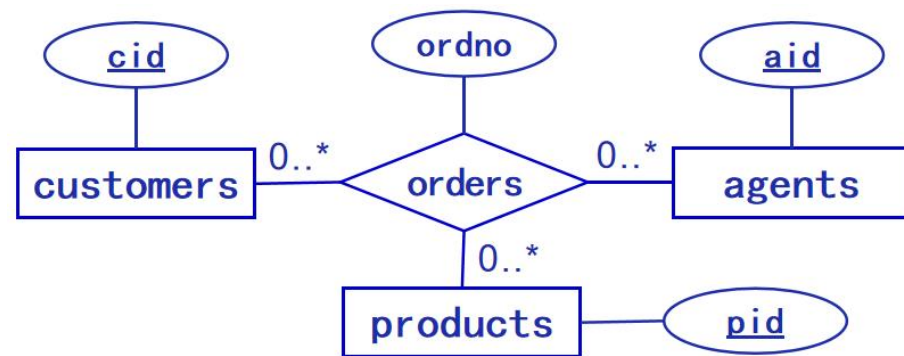
主讲(工号, 课程班号)

修读(学号, 课程班号, 成绩)

多元联系的转换

❑ 规则6：多个实体集之间的多元联系

- 每个实体集转换成一个关系模式
- 联系被单独转换成一个关系模式（参照多对多二元联系的转换）
 - 其属性包括：
 - a) 联系自身的属性
 - b) 参与联系的每个实体集对应关系的主码
- 多元联系转换得到的关系模式，其码一般由所有参与联系的实体集的主标识符联合组成（参照多对多二元联系的转换，例如：学生选课）
- 但是也可能存在变化，需要通过后续的规范化设计才能确认关系上的码。例如：图书借阅，商品零售

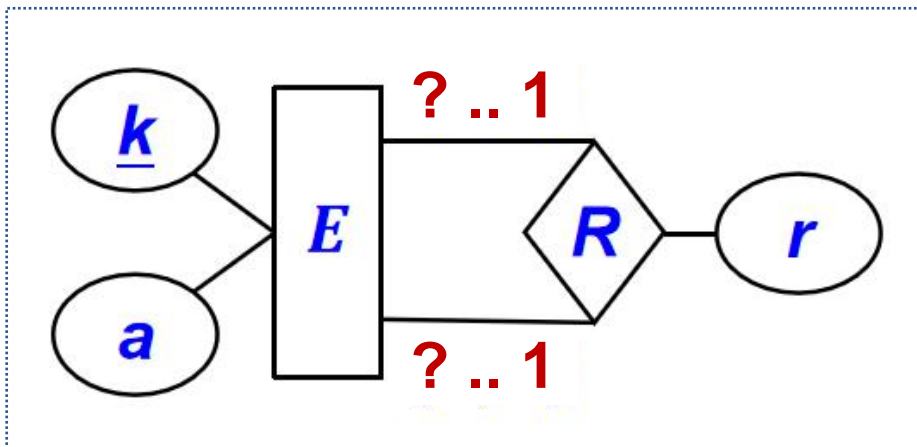


单个实体集内部联系的转换

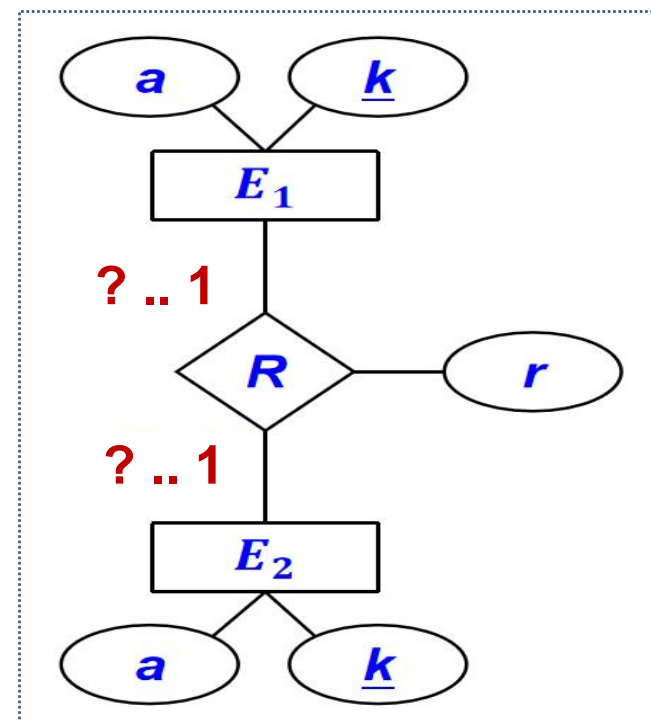
□ 规则7：单个实体集内部的联系

- 可参照二元联系、多元联系的转换规则
- 以单个实体集内部的二元联系为例：
 - ① 首先，将该联系转换成两个实体集之间的二元联系；
 - ② 再按照二元联系的处理方式转换成关系模式；
 - ③ 将两个实体集转换得到的关系模式合并为一个关系。

规则7-1：单个实体集内部的联系（一对一）



➤ 根据在联系 R 中所担当的角色，将实体集 E 划分为 E_1 和 E_2 两个实体集



□ 转换成三个关系模式

$E_1(k, a)$

$E_2(k, a)$

$R(k_1, k_2, r)$

其中：

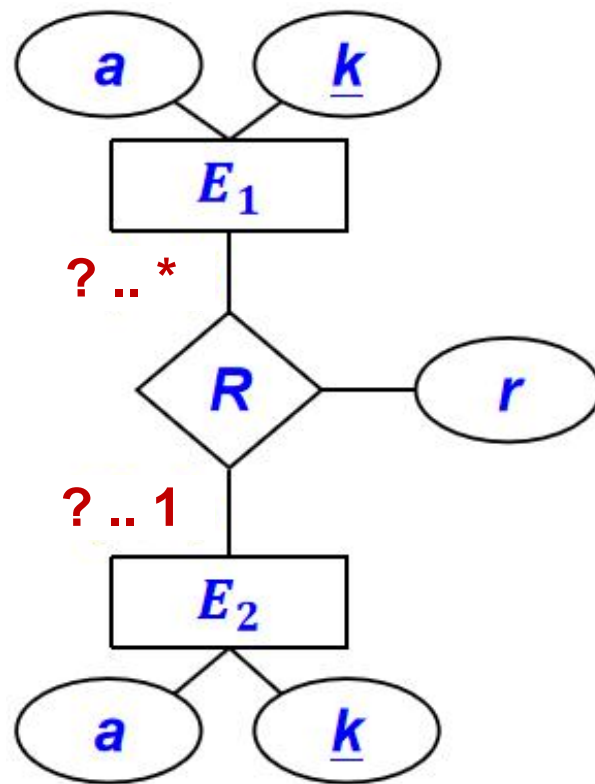
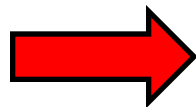
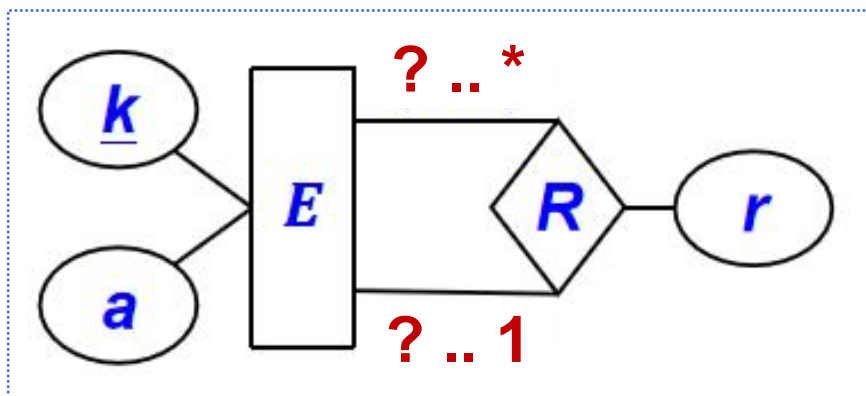
- k_1 和 k_2 是 R 的两个码
- k_1 和 k_2 也是 R 的两个外码，且 k_1 对应 E_1 的主码 k ， k_2 对应 E_2 的主码 k

➤ E_1 和 E_2 其实是同一个关系，联系 R 也可以被合并到 E_1 或 E_2 对应的关系中去，合并后的关系模式是： $E(k, a, k', r)$

□ 在最终方案中，必须区分清楚码 k 和外码 k' 在联系 R 中的角色！

规则7-2: 单个实体集内部的联系 (一对多)

- 根据在联系R中所担当的角色，将实体集E划分为 E_1 和 E_2 两个实体集



- 转换成两个关系模式 (联系R被合并到‘单值参与’端)

$E_1(k, a)$

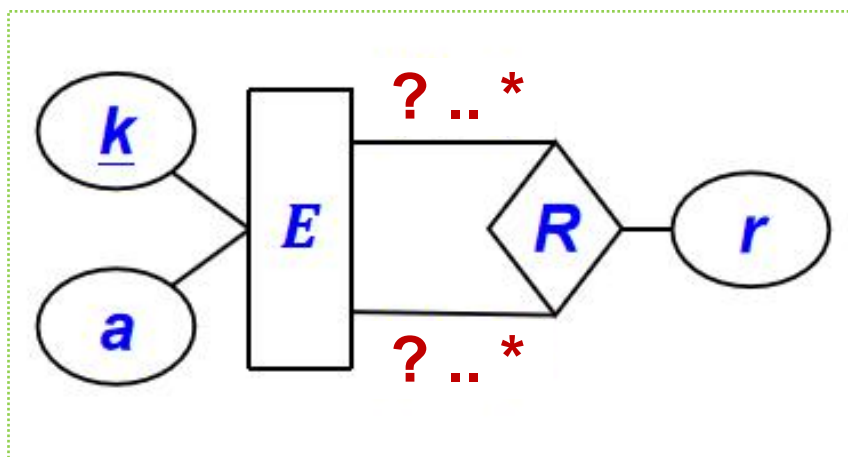
$E_2(k, a, k1, r)$

其中: $k1$ 对应 E_1 的主码 k

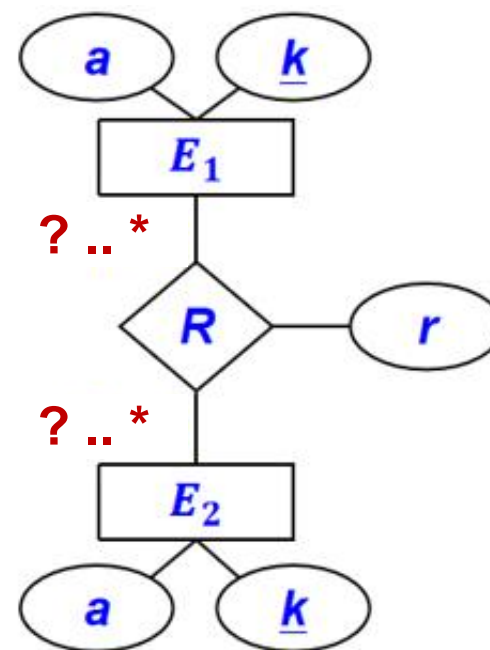
- 只需要保留关系 E_2 , 得到最终的模式设计方案是: $E(k, a, k', r)$

- 在最终方案中, 码 k 对应‘单值参与’端, 外码 k' 对应‘多值参与’端。

规则7-3：单个实体集内部的联系（多对多）



- 根据在联系R中所担当的角色，将实体集E划分为E₁和E₂两个实体集



□ 转换成三个关系模式

$E_1(k, a)$

$E_2(k, a)$

$R(k1, k2, r)$

其中：

- (k1, k2) 是R的主码
- k1和k2是关系R的两个外码，且k1对应E₁的主码k，k2对应E₂的主码k

- E₁和E₂只需保留一个，最终构成两个关系的设计方案：

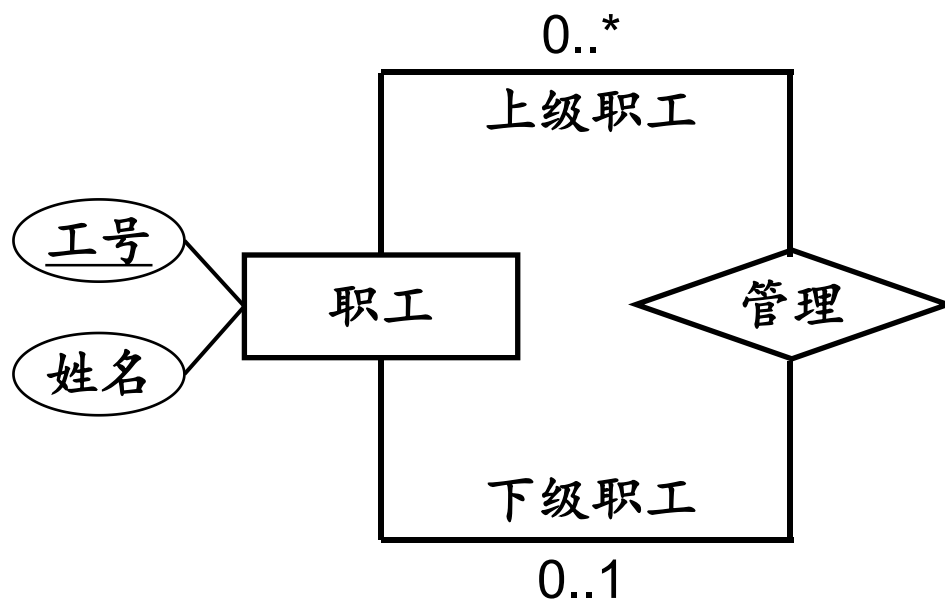
$E(k, a)$

$R(k1, k2, r)$

□ 在最终方案中，必须区分清楚关系R中的两个外码k1和k2在联系中的角色！

规则7：单个实体集内部联系的转换（示例）

□ 例：职工实体集内部的（直接）上下级联系



□ 根据规则7-2，可以将联系合并到职工实体集对应的关系中（‘工号’是码，‘上级职工工号’是外码）

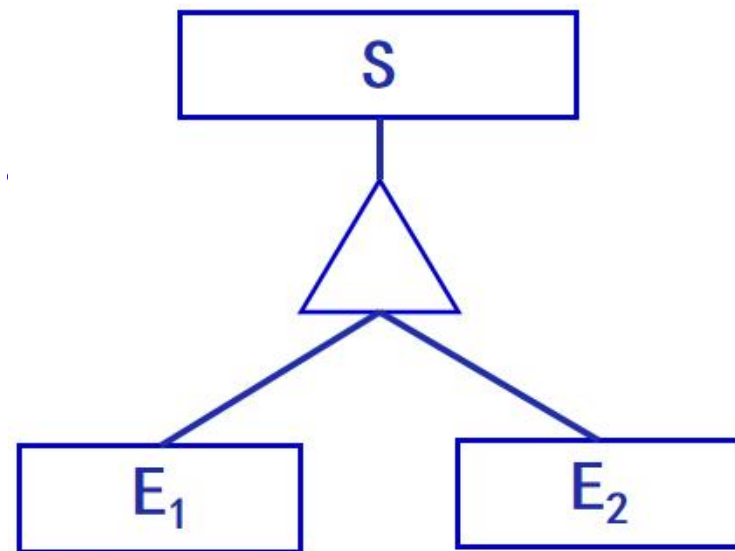
职工 (工号, 姓名,, 上级职工工号)

4. ISA联系的转换

❑ 假设超实体集 S 与其两个子实体集 E_1 和 E_2 构成如右图所示的继承关系（可重叠、部分特化）

❑ 其中：

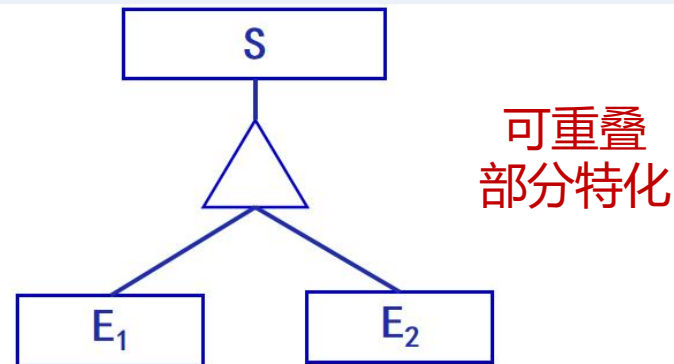
- 实体集 S 的属性有 $(k, A_1, A_2, \dots, A_n)$ ， k 是其标识属
- 实体集 E_1 的属性有 $(B_{11}, B_{12}, \dots, B_{1i})$
- 实体集 E_2 的属性有 $(B_{21}, B_{22}, \dots, B_{2j})$



❑ 有三种不同的转换方式可供选择，可分别转换为：1个关系、2个关系、3个关系

规则8: ISA联系的转换

- ❑ 超实体集**S**的属性有 $\{ \underline{k}, A_1, A_2, \dots, A_n \}$, **k**是其标识属性
- ❑ 子实体集**E₁**的属性有 $\{ B_{11}, B_{12}, \dots, B_{1i} \}$
- ❑ 子实体集**E₂**的属性有 $\{ B_{21}, B_{22}, \dots, B_{2j} \}$



- ① 转换方式1: 每一个实体集都将被单独转换为一个关系。每一个关系都含有超实体集(根结点)中的标识属性**k**, **k**是每一个关系的主码。

$S(\underline{k}, A_1, A_2, \dots, A_n)$

$E_1(\underline{k}, B_{11}, B_{12}, \dots, B_{1i})$

$E_2(\underline{k}, B_{21}, B_{22}, \dots, B_{2j})$

- ② 转换方式2: 只有最底层的叶子结点才会被转换为关系, 并继承从叶子结点到根结点的路径上所有超实体集中的所有属性。

$E_1(\underline{k}, A_1, A_2, \dots, A_n, B_{11}, B_{12}, \dots, B_{1i})$

$E_2(\underline{k}, A_1, A_2, \dots, A_n, B_{21}, B_{22}, \dots, B_{2j})$

- ③ 转换方式3: 所有实体集被转换为单个关系, 其中包含所有实体集中的属性。

$E(\underline{k}, A_1, A_2, \dots, A_n, B_{11}, B_{12}, \dots, B_{1i}, B_{21}, B_{22}, \dots, B_{2j})$

❑ 请仔细体会这三种不同转换方案之间的区别!

ISA 联系转换的例子 1

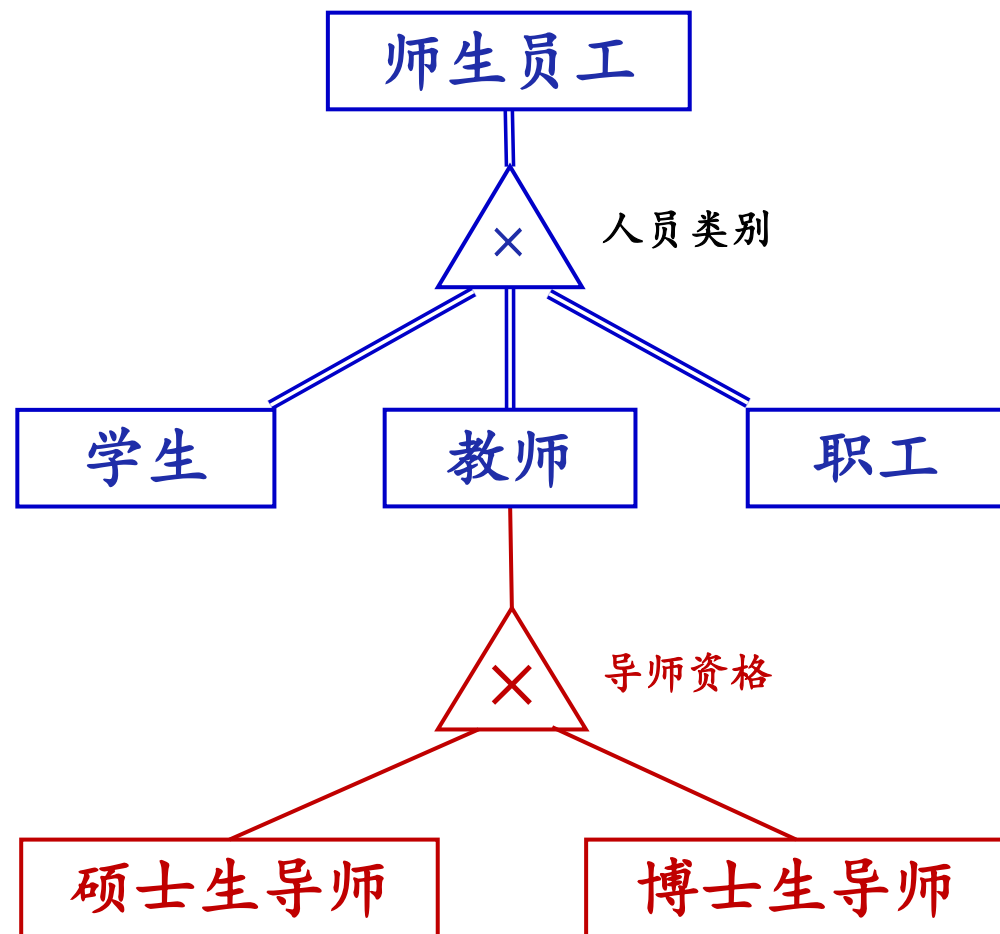
❑ 假设不考虑在职研究生

- 师生员工与学生、教师、职工之间是一个不相交、完全特化的**ISA**联系
- 可以采用前述的方式2，实现向关系的转换
- 转换成为**学生**、**教师**、**职工**这三个关系

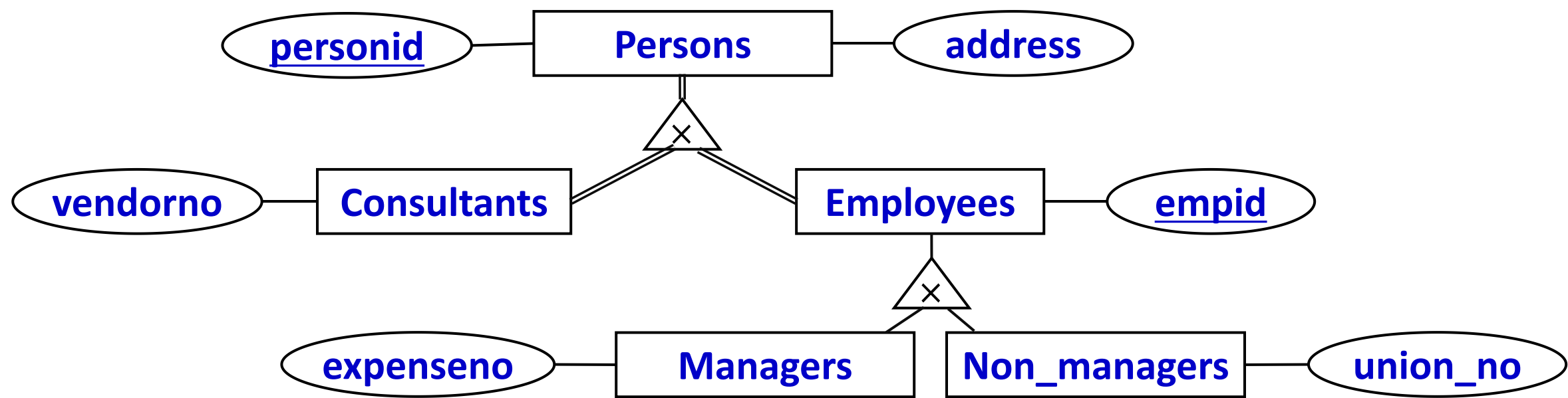
❑ 除此之外，其他的**ISA**联系，都采用方式1来实现向关系的转换

- 原**教师**关系不变
- 再增加两个新的关系：

- **硕士生导师** 和 **博士生导师**



ISA 联系转换的例子 2



❑ 在本例中，第一层ISA联系采用转换方式2，第二层ISA联系采用转换方式1

关系 Consultants

属性	属性级约束
personid	primary key
address	
vendorno	UNIQUE

关系 Employees

属性	属性级约束
personid	UNIQUE
address	
empid	primary key

关系 Managers

属性	属性级约束
empid	primary key
expenseno	UNIQUE

关系 Non_Managers

属性	属性级约束
empid	primary key
union_no	UNIQUE

- ❑ 在从概念数据模型（EE-R）到逻辑数据模型（关系模型）的转换过程中，还可能存在一些需要处理的问题。
 1. 命名与属性域的处理
 2. 非原子属性处理
 3. 联系的转换
 4. **ISA**联系的转换
 5. 规范化
 - 对转换得到的关系模式进行关系规范化设计，确保所有关系模式至少满足第三范式(3NF)
 6. **RDBMS**性能调整
 7. 约束条件设置
 8. 关系视图的设计

6. 关系数据库管理系统 (DBMS) 性能调整

➤ 为满足数据库系统在性能、存储空间等方面的要求及其它限制条件所做的模式调整与修改。包括：

(1) 逆规范化

- 减少关系的连接运算次数，提高系统性能

(2) 关系的分割

(3) 尽量使用快照

(2) 关系的分割

- 调整每个关系的大小，提高在单个关系上的数据存取效率。
- 常用分割方法：
 - 水平分割
 - 将一个关系的元组集合划分为若干个不相交的子集，每个子集对应一个子关系模式；
 - 垂直分割
 - 将一个关系模式纵向分解成若干个子关系模式。（不同于规范化设计中的模式分解）

(3) 尽量使用快照

➤ 快照(snapshot)

```
create snapshot <snapshot-name> as <select-statement>;
```

- 通过创建‘快照’，可以用一条**select**查询语句来为数据库中的某张表、或者为某个数据查询请求的执行结果创建一个不可修改的、持久化的数据备份。
- 在执行**create snapshot**命令时，**DBMS**将执行对应的查询语句**<select-statement>**，并将查询执行结果以‘表’的形式持久存储在数据库中。
- 在实际应用中，应用程序可以通过访问快照直接使用保存在数据库中的查询结果，不必再去执行对应的查询命令，有效提高了应用访问效率。特别适合于业绩统计、库存盘点、实时计费等应用场景。

➤ 快照的维护：**DBMS**提供了两种快照结果的更新方式

- 由**DBMS**周期性地自动刷新快照的结果数据
- 由用户通过执行快照刷新命令来手工刷新快照的结果数据

7. 约束条件的设置

- 完整性约束
- 安全性约束
- 数据类型约束
- 数据量的约束
- 重新定义每个表的候选键、主码及外码

8. 关系视图的设计

- 在关系模式基础上所设计的直接面向操作用户的视图，即用户的‘**外模式**’，它可以根据用户的需求随时定义或重新定义，一般 **RDBMS** 均通过‘视图’(view)来提供关系视图的功能。
- 关系视图的作用
 - 提供数据的逻辑独立性
 - 能满足不同用户的不同数据需求
 - 提供一定的数据安全、保密功能

□ 数据库逻辑设计的案例

- 案例1：图书借阅管理（复习思考题13）
- 案例2：考试监考安排管理（复习思考题14）
- 案例3：CBA篮球联赛信息管理（复习思考题15）
- 案例4：邮件信息管理（复习思考题16）
- 案例5：电子图书借阅管理（复习思考题17）
- 案例6：民用航班信息管理（复习思考题18）

（参考：数据库设计案例参考答案）